

Task 3 — Simple Linear Regression with Gradient Descent

Student Guide | scikit-learn fit() + Gradient Descent from Scratch

Task 2 (data cleaning / preprocessing) is already complete. This task has two parts that every student must complete:

1. Use scikit-learn's LinearRegression to fit a model on your data (the quick, library way).
2. Implement Gradient Descent by hand (no sklearn) to find the same parameters, so you understand what fit() is actually doing underneath.

Worked Example — iPhone Dataset

This guide uses a small `iphone.csv` (iPhone model number → Price) to demonstrate every step end-to-end. Students must repeat the same steps on their OWN dataset for submission.

1. The Sample Data — `iphone.csv`

A tiny dataset with one input feature (model number) and one target (price):

number (X)	iPhone name	Price (Y)
8	iPhone 8	499
10	iPhone X	600
11	iPhone 11	800
12	iPhone 12	900
13	iPhone 13	1000
14	iPhone 14	1299

Goal: fit a straight line $Y = m \cdot X + c$ so that, given a new model number, we can predict its price. Here m is the slope and c is the intercept — these are exactly the two parameters gradient descent will learn.

2. Part A — Fit the Model Using scikit-learn

This is the fast, production way: sklearn already implements the maths internally.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv('iphone.csv')
X = df[['number']]      # feature(s) as a 2-D array -> double brackets
y = df['Price']          # target as a 1-D array

model = LinearRegression()
model.fit(X, y)          # <-- this is the 'fit' step students must call
```

```
print('Slope (m):', model.coef_)
print('Intercept (c):', model.intercept_)
print('Prediction for model 15:', model.predict([[15]]))
```

Running this on `iphone.csv` gives:

$$m \approx 128.60 \quad c \approx -607.80 \quad \rightarrow \quad \text{Price} \approx 128.60 \times \text{number} - 607.80$$

This single `fit()` call is what students must replicate step-by-step in Part B using Gradient Descent, so they understand how `sklearn` finds `m` and `c` internally.

3. Part B — Implement Gradient Descent by Hand

Gradient Descent is an iterative algorithm that starts with a random guess for `m` and `c`, and repeatedly nudges them in the direction that reduces error, until the error stops improving much.

3.1 Step 1 — Define the Loss Function

The loss function measures how wrong the current line is. We use Mean Squared Error (MSE) — the average of the squared difference between actual price (`Y`) and predicted price ($\hat{Y}$):

$$L(m, c) = (1/n) \sum (Y_i - (m \cdot X_i + c))^2$$

Why squared? Squaring removes negative signs (so errors don't cancel out) and penalises large errors more heavily than small ones. Smaller `L` = better-fitting line. The whole job of gradient descent is: find the (`m`, `c`) that minimises `L`.

3.2 Step 2 — Find the Gradient w.r.t. `m` and `c`

The gradient tells us the slope of the loss surface — i.e. which direction increases the loss. We move opposite to it ('descent'). Taking the partial derivative of `L` with respect to each parameter:

$$\partial L / \partial m = -(2/n) \sum X_i \cdot (Y_i - (m \cdot X_i + c))$$

$$\partial L / \partial c = -(2/n) \sum (Y_i - (m \cdot X_i + c))$$

In code, if `error = predicted - actual`, these simplify to (sign flipped, same result):

```
dm = (2/n) * sum(error * X)    # gradient w.r.t. slope m
dc = (2/n) * sum(error)         # gradient w.r.t. intercept c
```

3.3 Step 3 — Update Rule (the 'Descent' step)

Each epoch, subtract a small fraction of the gradient from the current parameters. The fraction is the learning rate (`lr`), typically 0.0001 to 0.01:

$$m_{\text{new}} = m_{\text{old}} - lr \times (\partial L / \partial m) \quad c_{\text{new}} = c_{\text{old}} - lr \times (\partial L / \partial c)$$

3.4 Step 4 — Choosing the Number of Epochs

One epoch = one full pass of computing the loss, the gradients, and updating m and c once. Guidelines:

- Too few epochs → the line stops updating before it reaches the best fit (underfitted parameters).
- Too many epochs → wastes computation once the loss has already flattened out (diminishing returns).
- Practical rule: plot loss vs. epoch. Stop when the loss curve goes flat (converges) — typically when the loss changes by less than a very small threshold (e.g. < 0.0001) between epochs.
- If the loss increases or oscillates wildly, the learning rate is too high — reduce it.
- If the loss decreases extremely slowly, the learning rate is too low or more epochs are needed — increase lr slightly or scale/normalise features.

4. Full Gradient Descent Code (iPhone Example)

```
import pandas as pd
import numpy as np

df = pd.read_csv('iphone.csv')
X = df['number'].values.astype(float)
Y = df['Price'].values.astype(float)
n = len(X)

m, c = 0.0, 0.0          # 1. start with initial guesses
lr = 0.001                # 2. learning rate
epochs = 20000            # 3. number of iterations

loss_history = []
for i in range(epochs):
    Y_pred = m * X + c    # current prediction
    error = Y_pred - Y    # predicted - actual

    loss = (1/n) * np.sum(error ** 2) # MSE loss
    loss_history.append(loss)

    dm = (2/n) * np.sum(error * X) # gradient w.r.t. m
    dc = (2/n) * np.sum(error)     # gradient w.r.t. c

    m = m - lr * dm             # update m
    c = c - lr * dc             # update c

    if i % 4000 == 0:
        print(f'epoch {i}: loss={loss:.2f} m={m:.3f} c={c:.3f}')

print('Final parameters -> m:', m, ' c:', c)
```

4.1 What the Loss Actually Does Over Epochs

Running the code above on `iphone.csv` ($lr = 0.001$) produces this progression — notice loss falling and (m , c) converging towards sklearn's values ($m \approx 128.60$, $c \approx -607.80$):

Epoch	Loss (MSE)	m	c
0	791067.00	20.26	1.70
4000	11772.85	86.93	-121.39
8000	9179.28	95.61	-222.64
12000	7553.07	102.47	-302.81
16000	6533.40	107.91	-366.30
19999	5894.18	112.22	-416.56

Convergence tip

With this small, unscaled feature range (8–14) and $lr = 0.001$, convergence is slow — it takes roughly 150,000–200,000 epochs to fully match sklearn's exact values. Students should either (a) run more epochs, (b) increase the learning rate carefully, or (c) scale/normalise X before training — all three are valid ways to speed up convergence, and comparing them is a good exercise.

5. What Students Must Submit This Week

Using their OWN dataset, every student must apply Multiple Linear Regression only (this week's scope is Linear Regression, not classification models):

3. Load your own preprocessed dataset (from Task 2).
4. Select one target column (Y) and one or more feature columns ($X_1, X_2, X_3, \dots$).
5. Part A: Fit the model using sklearn's `LinearRegression().fit(X, y)`. Print the coefficients and intercept.
6. Part B: Implement Gradient Descent from scratch for your data (extend the m, c update rule to one weight per feature, plus a bias/intercept term). Print the loss curve and final learned parameters.
7. Compare: do the sklearn parameters and your gradient-descent parameters roughly match? Include a short note explaining any difference (usually learning rate / epochs).

If your dataset is a classification problem

This week's task is Linear Regression only. If your own dataset's target is categorical (classification), do not use it here — instead pick any other dataset with a numeric (continuous) target and apply Linear Regression to that. Classification models (Logistic Regression, etc.) are not part of this week's task.

5.1 Extending Gradient Descent to Multiple Features

For multiple linear regression, $Y = w_1X_1 + w_2X_2 + \dots + w_kX_k + c$. The loss function stays the same form (MSE), and each weight gets its own gradient and update rule:

$$\partial L / \partial w_j = -(2/n) \sum X_{ij} \cdot (Y_i - \hat{Y}_i) \quad w_{j_new} = w_{j_old} - lr \times (\partial L / \partial w_j)$$

In practice this is done with matrix/vector operations (NumPy) instead of separate variables per feature — happy to walk through this in the lab session if needed.

6. Quick Checklist Before Submission

- ☒ sklearn LinearRegression fit() run and coefficients printed.
- ☒ Gradient descent implemented from scratch (no sklearn) with your own loss/gradient/update code.
- ☒ Loss vs. epoch printed or plotted, showing convergence.
- ☒ Final (m, c) or (weights, c) from gradient descent compared with sklearn's values.
- ☒ Applied to your OWN dataset (not the iPhone example, which is for reference only).
- ☒ If your dataset is classification-only, a substitute regression dataset was used instead.